Advanced OCI Networking Debugging Guide

A Deep-Dive into Real Multi-Tier Debugging, Root Cause Analysis, and Network Theory

This document is a detailed, high-quality reconstruction of the entire 2■day debugging process we performed

while solving a complex OCI networking issue in a multi-tier architecture consisting of:

• Reverse Proxy (Public Subnet)

• App Server (Private Subnet)

• Oracle Cloud Virtual Cloud Network (VCN)

This guide includes every diagnostic step, why we ran each command, what each symptom meant, and how each

layer of the network stack contributes to the behavior we observed.

It is designed to serve as a long-term reference for cloud networking, Linux networking, and OCI-specific internals.

---

1. PROBLEM SUMMARY

The core issue:

The reverse-proxy instance could not reach the app-server private IP (10.0.2.x).

Error:

curl -v http://10.0.2.38

→ No route to host

Observations:

• Both servers were in same VCN.

• Correct subnets (public 10.0.1.0/24, private 10.0.2.0/24).

• Correct route tables and NSGs.

• NGINX was running on app-server.

• App-server could curl itself.

• Reverse-proxy could not ARP or SYN the app-server.

This created a highly unusual scenario suggesting a deeper SDN-level failure.

---

2. DEBUGGING METHODOLOGY – LAYERED APPROACH

We used a formal OSI-based debugging flow:

Layer 1: Physical (ignored – virtual cloud networking abstracts this)

Layer 2: ARP / Neighbor Discovery

Layer 3: Routing

Layer 4: TCP Handshake (SYN)

Layer 7: Application (NGINX)

Every diagnostic command fits one OSI layer.

---

3. LAYER 2 ANALYSIS – ARP & NEIGHBOR DISCOVERY

Key commands:

ip neigh

sudo tcpdump -n -i any host and arp

sudo tcpdump -n -i any tcp and host and port 80

Why:

Layer 2 decides who can physically “see” whom on the VCN overlay network.

Findings:

• Reverse-proxy never created ARP entries for app-server.

• App-server sometimes saw SYN packets from old reverse-proxy, but never the other way around.

• tcpdump showed outgoing SYN from reverse-proxy but 0 captured packets = means SYN never left hypervisor.

Conclusion:

Either routing or VNIC attachment broken.

---

4. LAYER 3 ANALYSIS – ROUTING TABLES

Command:

ip route show

Expected:

10.0.2.0/24 via 10.0.1.1 dev enp0s6

Findings:

Routing table was correct the entire time.

But packets still failed to leave the VM → meaning the **hypervisor rejected them before routing**.

---

5. OS FIREWALL TESTS

Commands:

sudo systemctl stop firewalld

sudo iptables -L -n -v

sudo nft list ruleset

getenforce

Why:

To ensure Linux wasn’t blocking outbound packets.

Findings:

• Firewalld disabled → no change

• SELinux disabled → no change

• iptables baseline rules only included OCI system services

• Nothing was blocking port 80

Conclusion:

NOT a Linux-level firewall issue.

---

6. rp_filter ANALYSIS – REVERSE PATH FILTERING

Commands:

cat /proc/sys/net/ipv4/conf/enp0s6/rp_filter

sudo sysctl -w net.ipv4.conf.all.rp_filter=0

Why:

Strict rp_filter drops packets when inbound/outbound interface mismatch occurs.

Result:

No change.

Conclusion:

Not rp_filter.

---

7. MTU DEBUGGING

Commands:

ip link show enp0s6

sudo nmcli connection modify mtu 1500

Why:

Incorrect MTU (e.g., 9000) on VCN overlay can drop packets silently.

Result:

After lowering MTU → no change.

Conclusion:

Not MTU mismatch.

---

8. ARP CACHE & NIC RELOAD

Commands:

sudo ip neigh flush all

sudo nmcli device reapply

sudo nmcli device disconnect/connect

Why:

To ensure no stale neighbor entries.

Result:

Still no SYN/ARP leaving interface.

---

9. THE CRITICAL DISCOVERY – METADATA SERVICE

OCI's metadata service is the ONLY 100% reliable source of truth about:

• Actual subnet

• Actual VNIC state

• Actual routing domain

Command:

sudo oci-metadata -j | jq '.vnics[0].subnetCidrBlock'

This revealed the **root cause**:

App-server output BEFORE recreation:

""

Reverse-proxy output:

"10.0.1.0/24"

Meaning:

The app-server’s VNIC was CREATED WITHOUT A SUBNET.

Effects:

• No private routing

• No ARP allowed

• No packet forwarding

• No route propagation

• Packets dropped at hypervisor BEFORE leaving VM

This EXACT issue is known on:

• A1.Flex ARM

• Restricted student/free tier

• EU-FRANKFURT region

OCI console UI may display correct subnet, but metadata reveals the REAL state.

---

10. RECREATION OF APP-SERVER

After deleting and re-creating the server:

"10.0.2.0/24"

Now correct.

This fixed the VNIC’s attachment to the subnet.

Immediately afterward → connectivity restored.

---

11. WHY DID THIS HAPPEN?

This is an OCI backend bug involving:

• ARM (A1.Flex) instances

• Student/free-tier accounts

• Frankfurt region

• Subnet attachment race conditions

• SDN plane fragmentation

OCI sometimes provisions VNICs WITHOUT binding them properly to the SDN fabric.

The console UI shows them as attached → but metadata reveals they are “ghost-attached”.

Symptoms:

• No ARP

• No route

• No SYN

• No tcpdump output

• No metadata subnet

Only fix:

Terminate → Recreate instance.

---

12. LESSONS LEARNED

---

1. OCI Metadata > Console UI

2. If routing looks correct but traffic never leaves → suspect VNIC corruption

3. Layered debugging finds root cause even in cloud SDN failures

4. Cloud networking is not infallible – sometimes the platform is wrong

5. Understanding ARP, routing, rp_filter, MTU, and tcpdump is essential for real-world debugging

6. Multi-hour persistence builds deep skill, not wasted effort

This is EXACTLY how senior cloud/network engineers debug real incidents.

---

13. FINAL SUMMARY

Root Cause:

The app-server VNIC was created with a NULL subnetCidrBlock.

Fix:

Recreate the app-server VM.

Outcome:

Networking restored.

Reverse-proxy → app-server works.